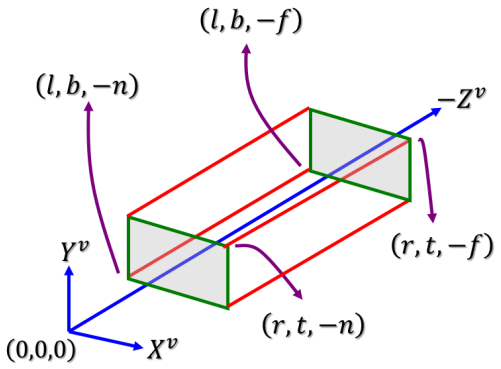
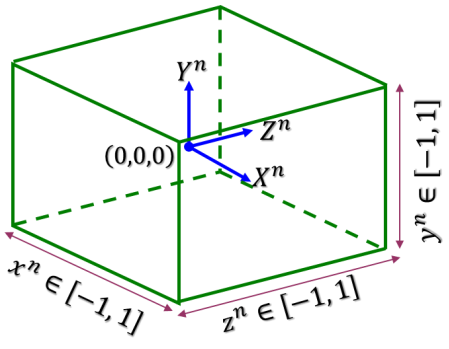
Orthographic Projection Transformation: Derivation

3D graphics pipes compute projections [both orthographic parallel and perspective] in two steps: ***shoot*** and ***print***. The ***shoot*** step is similar to capturing an image of the scene on film using an analog camera. Instead of capturing the image on a 2D film, the graphics pipe captures the image of the scene in a normalized device coordinates (NDC) box. The NDC box is the 3D equivalent of 2D film. The ***print*** step involves the proportional scaling of the NDC image to fit the rectangular *viewport*. This step is analogous to printing the film on paper. This document describes the mathematical derivations of the ***shoot*** step.

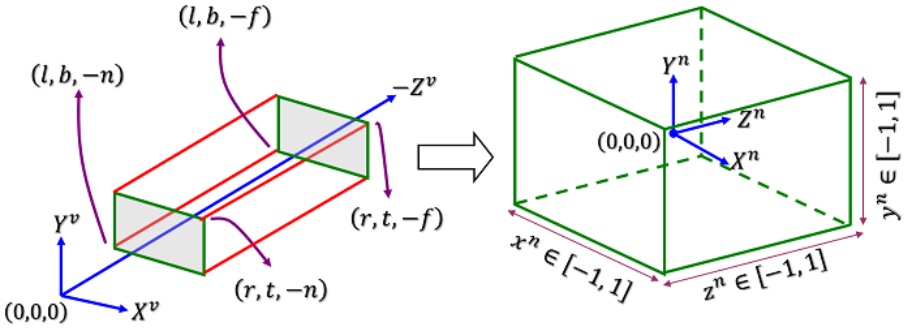
In orthographic projection, a viewing box is defined in the view coordinate system. The viewing box determines the portion of the scene that is visible to the camera. As shown in the following figure, the box's sides are aligned along X^v, Y^v, and Z^v axes. The box spans along X^v-axis from l to r, along Y^v-axis from b to t, and along Z^vaxis from $-n$ [location of *near* plane] to $-f$ [location of *far* plane]. Note the quirk in OpenGL: the view vector - the direction along which the camera is looking in the world coordinate system - is mapped to the view coordinate system's $-Z^v$-axis.



In orthographic projection, the ***shoot*** step transforms the contents of the viewing box to a NDC box. As shown in the following figure, the NDC box is defined in a left-handed coordinate system with its center at the origin and having width, height, and depth of size 2.



The following figure represents the mapping of the contents of the viewing box in view coordinate system to the NDC box in NDC coordinates.



How is this transformation computed? We can see that a sequence of scales and translations will map the viewing parallelepiped [left-hand side of above figure] to be equivalent to the NDC box [right-hand side of above figure]. Since matrix multiplication is not commutative, not only must the scalings and translations be correctly computed, but these transformations must be concatenated in the correct order. Consider the problem of taking a value in input range $I_{in} \in [-13, 28]$ and finding its corresponding value in output range $I_{out} \in [16, 77]$. First, I_{in} must be centered by a translation $-\frac{15}{2}$, next the transformed values in I_{in} must be scaled by $\frac{1}{41}$ so that they're transformed to range $\left[-\frac{1}{2}, \frac{1}{2}\right]$, followed by another scaling by 61 to match the output range centered at 0, and finally concluding with translation $\frac{93}{2}$ to displace the centered values to $I_{out} \in [16, 77]$.

Rather than tripping ourselves with computing translations and scalings in 3D and concatenating these matrices together in the correct order, we'll use a more direct method that will be referred to as an ***exemplary method*** – *exemplary* because we'll be using *examples* of values to obtain the mappings. We begin by dividing the (big) 3D transformation problem into three simpler but similar 1D problems – one problem per axis. If we can solve one of these smaller 1D problems, then we can use its solution as a template for solving the other two similar problems. We will then reconstitute the answers to these three simple problems as the solution to computing the orthographic projection matrix.

Let's begin by solving the problem of mapping input values $x^v \in [l, r]$ along X^v-axis to output values $x^n \in [-1, 1]$ along X^n-axis. Values in the output range can be obtained by an initial offset, a sequence of simple scalings followed by an offset. The net cumulative effect of these scales and offsets can be written as a linear equation:

$$x^n = ax^v + b \tag{1}$$

where constants a and b are the unknowns. It is easiest to compute these unknowns by specifying the equation in terms of two example input values l and r and their associated desired output values -1 and 1, respectively. That is,

$$\begin{aligned} l &\longmapsto -1 \\ r &\longmapsto 1 \end{aligned} \tag{2}$$

Substituting the two examples from equation (2) into equation (1) leads to the following two equations:

$$-1 = al + b \tag{3}$$

$$1 = ar + b \tag{4}$$

We compute the unknown cumulative scaling value represented by a by subtracting equation (4) from equation (3) :

$$
\begin{aligned}
-2 &= a(l - r) \\
\implies a &= \frac{2}{r - l}
\end{aligned}
\tag{5}
$$

The unknown cumulative translation value represented by b is computed by replacing a's result from equation (5) into equation (3):

$$
\begin{aligned}
-1 &= \frac{2l}{r - l} + b \\
\implies b &= -\frac{r + l}{r - l}
\end{aligned}
\tag{6}
$$

Using the results from equations (5) and (6), we can now write the mapping of input values $x^v \in [l, r]$ along X^v-axis to output values $x^n \in [-1, 1]$ along X^n-axis as:

$$
x^n = \frac{2}{r - l} x^v - \frac{r + l}{r - l} \tag{7}
$$

We repeat this exemplary method to map input values $y^v \in [b, t]$ along Y^v-axis to output values $y^n \in [-1, 1]$ along Y^n-axis:

$$
\begin{gathered}
y^n = cy^v + d \\
\\
b \longrightarrow -1 \\
t \longrightarrow 1 \\
\\
\begin{aligned}
-1 &= cb + d \\
1 &= ct + d
\end{aligned} \\
\\
\begin{aligned}
\implies c &= \frac{2}{t - b} \\
\implies d &= -\frac{t + b}{t - b}
\end{aligned} \\
\\
\implies y^n = \frac{2}{t - b} y^v - \frac{t + b}{t - b}
\end{gathered}
\tag{8}
$$

Finally, the exemplary method is used to map input values $z^v \in [-n, -f]$ along Z^v-axis to output values $z^n \in [-1, 1]$ along Z^n-axis. Why are the values of the near and far plane distances negated? Recall the quirk in OpenGL's view coordinate system that orients the view vector along $-Z^v$-axis. OpenGL introduced another quirk by making users specify positive values for the distances of near and far planes in (the now deprecated) glOrtho function that computes the orthographic projection transformation matrix. Users were required to specify positive values for near and far plane distances such that $f > n > 0$. But since near and far planes are located along $-Z^v$ axis, z^v values in viewing box will be $z^v \in [-n, -f]$.

The exemplary method is repeated to map input values $z^v \in [-n, -f]$ along Z^v-axis to output values $z^n \in [-1, 1]$ along Z^n-axis:

$$z^n = gy^v + h$$

$$\begin{aligned} -n &\longrightarrow -1 \\ -f &\longrightarrow 1 \end{aligned}$$

$$\begin{aligned} -1 &= -gn + h \\ 1 &= -gf + h \end{aligned}$$

$$\begin{aligned} \implies g &= -\frac{2}{f-n} \\ \implies h &= -\frac{f+n}{f-n} \end{aligned}$$

$$\implies z^n = -\frac{2}{f-n} z^v - \frac{f+n}{f-n} \tag{9}$$

Let's aggregate equations (7), (8), and (9):

$$x^n = \frac{2}{r-l} x^v - \frac{r+l}{r-l}$$

$$y^n = \frac{2}{t-b} y^v - \frac{t+b}{t-b}$$

$$z^n = -\frac{2}{f-n} z^v - \frac{f+n}{f-n}$$

Rewriting these equations as linear functions of x^v, y^v, and z^v values:

$$\begin{aligned} x^n &= \left(\frac{2}{r-l}\right) x^v + (0) y^v + (0) z^v - \frac{r+l}{r-l} \\ y^n &= (0) x^v + \left(\frac{2}{t-b}\right) y^v + (0) z^v - \frac{t+b}{t-b} \\ z^n &= (0) x^v + (0) y^v + \left(-\frac{2}{f-n} z^v\right) - \frac{f+n}{f-n} \\ 1 &= (0) x^v + (0) y^v + (0) z^v + 1 \end{aligned} \tag{10}$$

Equation (10) represents a system of four linear equations which can be expressed as a matrix equation of the form

$$\mathcal{M}_{ortho} P^v = P^n$$

where $\mathcal{M}_{ortho}$ represents the 4×4 orthographic projection matrix, $P^v = (x^v, y^v, z^v)$ represents a view coordinate system point in the viewing parallelepiped, and $P^n = (x^n, y^n, z^n)$ represents the corresponding NDC point in the $2 \times 2 \times 2$ NDC box.

The orthographic projection matrix $\mathcal{M}_{ortho}$ can be written as:

$$\mathcal{M}_{ortho} = \begin{bmatrix} \frac{2}{r-l} & 0 & 0 & -\frac{r+l}{r-l} \\ 0 & \frac{2}{t-b} & 0 & -\frac{t+b}{t-b} \\ 0 & 0 & -\frac{2}{f-n} & -\frac{f+n}{f-n} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Orthographic transformation matrix in GLM

The `glm::ortho` function creates the orthographic parallel viewing volume shown in the first picture. If l and r specify the coordinates for the left and right vertical clipping planes; b and t specify the coordinates for the bottom and top horizontal clipping planes; n and f specify the distances to the near and far depth clipping planes, we use `glm::ortho` to define a matrix for an orthographic parallel viewing volume:

```
glm::mat4 ortho_mtx = glm::ortho(l, r, b, t, n, f);
```

We can directly use our derivation of the orthographic projection matrix to define an orthographic parallel viewing volume using column-major layout:

```
glm::mat4 ortho_mtx(
  glm::vec4(2.f/(r-l),    0.f,          0.f,          0.f),
  glm::vec4(0.f,          2.f/(t-b),    0.f,          0.f),
  glm::vec4(0.f,          0.f,          -2.f/(f-n),   0.f),
  glm::vec4(-(r+l)/(r-l), -(t+b)/(t-b), -(f+n)/(f-n), 1.f));
```